

Regex Class

(System.Text.RegularExpressions.Regex)

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>Regex.IsMatch(input, pattern)</code>	Checks if regular expression finds a match anywhere within the input string (returns bool) EXAMPLE: <code>bool ok = Regex.IsMatch("Order-1234", @"^Order-\d+\$");</code>
<code>Regex.IsMatch(input, pattern, options, timeout)</code>	Checks match with RegexOptions and TimeSpan timeout to protect against catastrophic backtracking (ReDoS) EXAMPLE: <code>Regex.IsMatch(text, @"\b[A-Z]+\b", RegexOptions.IgnoreCase, TimeSpan.FromMilliseconds(250));</code>
<code>Regex.Match(input, pattern)</code>	Searches input string for first occurrence of regular expression; returns Match object EXAMPLE: <code>Match m = Regex.Match("id: 4209", @"\d+");</code>
<code>Regex.Matches(input, pattern)</code>	Searches input string for all occurrences of regular expression; returns MatchCollection EXAMPLE: <code>MatchCollection mc = Regex.Matches("10, 20, 30", @"\d+");</code>
<code>Regex.Replace(input, pattern, replacement)</code>	Replaces all matched substrings with specified replacement string (supports \$1, \${name}, \$&) EXAMPLE: <code>string res = Regex.Replace("hello world", @"\s+", " ");</code>
<code>Regex.Replace(input, pattern, matchEvaluator)</code>	Replaces matches using a MatchEvaluator delegate / lambda function for dynamic substitutions EXAMPLE: <code>Regex.Replace("cat and dog", @"\b\w+\b", m => m.Value.ToUpper());</code>
<code>Regex.Split(input, pattern)</code>	Splits input string into an array of substrings at positions defined by regex match EXAMPLE: <code>string[] parts = Regex.Split("apple, banana; cherry", @"[,;]\s*");</code>
<code>Regex.Count(input, pattern, options)</code>	Counts total number of non-overlapping matches without allocating Match objects (.NET 7+) EXAMPLE: <code>int count = Regex.Count("banana", "an"); // 2</code>
<code>Regex.EnumerateMatches(span, pattern)</code>	Zero-allocation high-performance ref struct enumerator over ReadOnlySpan<char> (.NET 7+) EXAMPLE: <code>foreach (ValueMatch vm in Regex.EnumerateMatches(span, pattern)) { ... }</code>
<code>Regex.Escape(str) / Regex.Unescape(str)</code>	Escapes regex metacharacters with backslashes or unescapes any escaped characters in a string EXAMPLE: <code>Regex.Escape("user[1].id") → @"user\[1]\.id"</code>
<code>new Regex(pattern, options, timeout)</code>	Creates an instance of Regex with compiled or interpreted options and optional match timeout EXAMPLE: <code>var rx = new Regex(@"\d+", RegexOptions.Compiled, TimeSpan.FromSeconds(1));</code>

Source Generators & Modern C# (.NET 7/8/9+)

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>[GeneratedRegex("pattern")]</code>	C# 11 / .NET 7+ compile-time source generator attribute on partial Regex method (Zero startup overhead, AOT-friendly) EXAMPLE: <code>[GeneratedRegex(@"^w+@w+\.w+\$", RegexOptions.IgnoreCase)]</code> <code>private static partial Regex EmailRegex();</code>
<code>[GeneratedRegex("pattern", options, matchTimeoutMilliseconds: 500)]</code>	Generates C# source code for regex matching engine at compile time with timeout safeguard EXAMPLE: <code>[GeneratedRegex(@"\d+", RegexOptions.None, 200)]</code> <code>private static partial Regex Numbers();</code>
<code>RegexOptions.NonBacktracking</code>	.NET 7+ engine using linear time DFA/NFA matching algorithms to guarantee O(n) runtime and eliminate ReDoS vulnerabilities EXAMPLE: <code>new Regex(@"^(a+)+\$", RegexOptions.NonBacktracking)</code>
<code>@"pattern"</code>	Verbatim string literal — backslashes are treated literally, avoiding double-escaping in C# EXAMPLE: <code>@"\d{4}-\d{2}-\d{2}"</code> instead of <code>"\\d{4}-\\d{2}-\\d{2}"</code>
<code>"""pattern"""</code>	Raw string literal (C# 11+) — allows quotes and unescaped characters across multiple lines without escape sequences EXAMPLE: <code>var pat = """^[a-z0-9' _]+\$""";</code>

Match, Group, & Capture Object Model

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>match.Success</code>	Boolean property indicating whether the regex match succeeded EXAMPLE: <code>if (m.Success) { ... }</code>
<code>match.Value</code> / <code>match.ValueSpan</code>	The matched substring value (or <code>ReadOnlySpan<char></code> in .NET 7+ for zero allocation) EXAMPLE: <code>m.Value</code> → "2026"
<code>match.Index</code> / <code>match.Length</code>	The zero-based start index and character length of the matched substring in original input EXAMPLE: <code>m.Index</code> → 12, <code>m.Length</code> → 4
<code>match.NextMatch()</code>	Searches the input string for the next match starting after the previous match ended EXAMPLE: <code>while (m.Success) { ...; m = m.NextMatch(); }</code>
<code>match.Groups[index]</code> / <code>match.Groups[name]</code>	Accesses captured groups by 0-based integer index (<code>Groups[0]</code> is entire match) or named group string key EXAMPLE: <code>string year = m.Groups["year"].Value;</code>
<code>group.Success</code> / <code>group.Value</code>	Indicates if an optional capturing group participated in the match and retrieves its captured value EXAMPLE: <code>if (m.Groups["ext"].Success) { ... }</code>
<code>group.Captures</code>	Collection of all individual Capture substrings for repeated capture groups (e.g. <code>(\w+)+</code>) EXAMPLE: <code>foreach (Capture c in m.Groups["word"].Captures) { ... }</code>
<code>capture.Index</code> / <code>capture.Length</code> / <code>capture.Value</code>	Zero-based starting position, length, and substring captured by a single repetition capture EXAMPLE: <code>c.Index</code> → 5, <code>c.Length</code> → 3, <code>c.Value</code> → 'foo'
<code>match.Result(replacementPattern)</code>	Expands replacement pattern string (<code>\$1</code> , <code>\${name}</code> , <code>\$&</code>) in context of this match EXAMPLE: <code>m.Result("\${first} \${last}")</code> → "Jane Doe"

RegexOptions Flags & Modifiers

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>RegexOptions.IgnoreCase</code>	Enables case-insensitive matching (inline syntax: <code>(?i)...</code> <code>(?-i)</code>) EXAMPLE: <code>Regex.IsMatch("ABC", "abc", RegexOptions.IgnoreCase) → true</code>
<code>RegexOptions.Multiline</code>	Changes <code>^</code> and <code>\$</code> to match at the beginning and end of each line instead of whole string (inline: <code>(?m)</code>) EXAMPLE: <code>Regex.Matches(lines, @"^#\s+", RegexOptions.Multiline)</code>
<code>RegexOptions.Singleline</code>	Dot-all mode: allows the dot <code>.</code> character to match every character, including newline <code>\n</code> (inline: <code>(?s)</code>) EXAMPLE: <code>Regex.Match(html, @"<div>.*</div>", RegexOptions.Singleline)</code>
<code>RegexOptions.ExplicitCapture</code>	Only captures explicitly named groups <code>(?<name>...)</code> ; unnamed parentheses <code>(...)</code> act as non-capturing (inline: <code>(?n)</code>) EXAMPLE: <code>new Regex(@"(abc)(?<num>\d+)", RegexOptions.ExplicitCapture)</code>
<code>RegexOptions.IgnorePatternWhitespace</code>	Extended/verbose mode: ignores unescaped whitespace and enables line comments with <code>#</code> (inline: <code>(?x)</code>) EXAMPLE: <code>@"\d{4} # Year\n\d{2} # Month"</code>
<code>RegexOptions.Compiled</code>	Compiles regular expression to MSIL bytecode at runtime for faster execution at cost of startup time EXAMPLE: <code>new Regex(@"\w+", RegexOptions.Compiled)</code>
<code>RegexOptions.NonBacktracking</code>	.NET 7+ non-backtracking engine with guaranteed linear-time matching; cannot be combined with <code>RightToLeft</code> , <code>ECMAScript</code> EXAMPLE: <code>new Regex(@"[a-z0-9._%+]+@[a-z0-9.-]+\.[a-z]{2,}", RegexOptions.NonBacktracking)</code>
<code>RegexOptions.RightToLeft</code>	Searches input string from right to left instead of left to right EXAMPLE: <code>new Regex(@"\d+", RegexOptions.RightToLeft).Match("10 20 30").Value → "30"</code>
<code>RegexOptions.CultureInvariant</code>	Ignores cultural language differences in string casing comparisons (uses invariant culture) EXAMPLE: <code>RegexOptions.IgnoreCase RegexOptions.CultureInvariant</code>
<code>RegexOptions.ECMAScript</code>	Enables ECMAScript-compliant behavior (can be combined only with <code>IgnoreCase</code> , <code>Multiline</code> , <code>Compiled</code>) EXAMPLE: <code>RegexOptions.ECMAScript RegexOptions.IgnoreCase</code>

.NET Specific Syntax & Advanced Constructs

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>(?<name>pattern) / (?'name'pattern)</code>	Named capture group — captures matched subexpression and assigns it a group name EXAMPLE: <code>(?<area>\d{3})-(?<local>\d{4})</code>
<code>(?<close-open>pattern)</code>	Balancing group definition (unique to .NET!) — deletes open group and captures interval between nested pairs EXAMPLE: <code>^[^()]*(((?'Open'\()[^()]*)+((?'Close-Open'\')[^()]*+)*?(?Open)(?!))\$</code>
<code>(?(name)yes no) / (?(1)yes no)</code>	Conditional matching — if named/numbered group matched, matches 'yes' pattern, else matches 'no' pattern EXAMPLE: <code>^(?<paren>\()(?\d{3})(?paren)\ -)(\d{4})\$</code>
<code>(?(?=pattern)yes no)</code>	Conditional matching with zero-width lookahead assertion as test condition EXAMPLE: <code>(?(?=\d{2}-)\d{2}-\d{2} \d{4})</code>
<code>(?im-s:pattern)</code>	Scoped inline flag modifiers — enables 'i' and 'm' and disables 's' only within enclosed subexpression EXAMPLE: <code>(?i:hello) (?-i:WORLD)</code>
<code>(?<=pattern) / (?<!pattern)</code>	Lookbehind assertions — .NET fully supports variable-length and non-fixed-width lookbehinds EXAMPLE: <code>(?<=[a-z]+):\d+</code>
<code>\p{L} / \p{N} / \p{P} / \p{Lu}</code>	Unicode general categories: Letter (<code>\p{L}</code>), Number (<code>\p{N}</code>), Punctuation (<code>\p{P}</code>), Uppercase Letter (<code>\p{Lu}</code>) EXAMPLE: <code>\p{Lu}\p{Ll}+</code>
<code>\p{IsBasicLatin} / \p{IsCyrillic}</code>	Unicode named block matching (e.g. <code>\p{IsGreek}</code> , <code>\p{IsArabic}</code> , <code>\p{IsDevanagari}</code>) EXAMPLE: <code>\p{IsBasicLatin}+</code>
<code>\$1, \${name}, \$&, \$`, \$', \$+</code>	Replacement tokens: numbered group (<code>\$1</code>), named group (<code>\${name}</code>), full match (<code>\$&</code>), before (<code>\$`</code>), after (<code>\$'</code>), last group (<code>\$+</code>) EXAMPLE: <code>Regex.Replace("John Smith", @"(?<first>\w+) (?<last>\w+)", "\${last}, \${first}")</code>
<code>\G</code>	Matches at the position where the previous match ended (contiguous matching) EXAMPLE: <code>\G\d{2}</code>